

Geospatial Query System with Natural Language Processing and GIS Integration

L.Y. Project Report

Submitted in partial fulfillment of the requirements of the Degree of Bachelor of Technology in Artificial Intelligence & Data Science

by

Riddhesh Chaudhary
Arpit Balmiki
Chirayu Vyas

Dr. Vaishali Wadhe



Department of Artificial Intelligence & Data Science

K.J. Somaiya Institute of Technology

Sion Ayurvihar Complex, Eastern Express Highway, Everard Nagar, Sion East, Mumbai-400022

2026

This page is intentionally left blank

This page is intentionally left blank



Geospatial Query System with Natural Language Processing and GIS Integration

L.Y. Project Report

Submitted in partial fulfillment of the requirements of the Degree of
Bachelor of Technology in Artificial Intelligence & Data Science

by

Riddhesh Chaudhary
Arpit Balmiki
Chirayu Vyas

Dr. Vaishali Wadhe



Department of Artificial Intelligence & Data Science

K.J. Somaiya Institute of Technology

Sion Ayurvihar Complex, Eastern Express Highway, Everard Nagar, Sion East, Mumbai-400022

2026

CERTIFICATE

This is to certify that the project entitled "Geospatial Query System with Natural Language Processing and GIS Integration" is a bona fide work of Riddhesh Chaudhary, Arpit Balmiki, and Chirayu Vyas, submitted to the University of Mumbai in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence & Data Science.

Dr. Vaishali Wadhe
Dean-Industry & Institute Interaction Cell
Dept. of AI&DS Engineering

.....(sign)
Dr. Milind Nemade
Head of Department

.....(sign)
Dr. Vivek Sunnapawar
Principal

(College Seal)

Place:

Date:

PROJECT APPROVAL FOR LAST YEAR

This project report entitled *Geospatial Query System with Natural Language Processing and GIS Integration* by

Riddhesh Chaudhary [seat no.] (Exam Seat No.)

Arpit Balmiki 26257007 (Exam Seat No.)

Chirayu Vyas [seat no.] (Exam Seat No.)

is approved for the Degree of Bachelor of Technology in Artificial Intelligence & Data Science.

Examiners

1.
(Internal Examiner Name and Sign)

2.
(External Examiner Name and Sign)

Date:

Place:

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Riddhesh Chaudhary _____

Arpit Balmiki _____

Chirayu Vyas _____

(Signature)

Date:

ACKNOWLEDGEMENT

To project guide Dr. Vaishali Wadhe,

We would like to express our sincere gratitude to our project guide, Dr. Vaishali Wadhe, for the guidance, encouragement and constructive feedback given at every stage of this project.

We are thankful to the Head of the Department, Dr. Milind Nemade, and to the Principal, Dr. Vivek Sunnapawar, for their support and for providing the facilities needed to carry out this work. We also thank the faculty members and the staff of the Department of Artificial Intelligence and Data Science, and K.J. Somaiya Institute of Technology, for their help during the project.

This project relies on open data and open-source software, and we gratefully acknowledge the OpenStreetMap contributors and the communities behind PostgreSQL, PostGIS, Martin, MapLibre, OSMnx, FastAPI and React.

Finally, we thank our families and friends for their patience and support.

Riddhesh Chaudhary

Arpit Balmiki

Chirayu Vyas

ABSTRACT

Choosing where to open a business, or judging whether a neighbourhood is safe, well connected and affordable, normally means days or weeks of manual surveys across maps, property listings and reports. Geographic Information Systems can answer such questions, but they demand skill in spatial data and query languages. The large language models (LLMs) that could make them accessible tend to invent place names and coordinates when they answer directly, which makes their output impossible to verify.

This project presents a natural-language geospatial query and decision-support system built on a spatial database. A user types a question in plain English, such as “hospitals within 1 km of a metro station”. An LLM pipeline first checks that the question is a clear spatial query and then writes a PostGIS query against a documented description of the database schema. The query is validated and executed under a read-only database role, and if it fails or returns nothing, the model is asked, within fixed limits, for a corrected query. The rows are returned as a GeoJSON layer drawn on an interactive MapLibre map, together with a short explanation and the exact SQL that was run, so that every answer can be checked.

The system runs on PostgreSQL 16 with PostGIS 3.4, serves vector tiles with Martin, and has a FastAPI backend and a React frontend. A loading pipeline built on OpenStreetMap and OSMnx fills the database for any city polygon. The deployed instance holds Mumbai, with 48 administrative boundaries, 16,800 road segments, 2,200 transit stops, 10,800 businesses and points of interest, a 76,000-row house-price dataset and area-level safety scores for 40 neighbourhoods. Four decision tools use the same data: a site report for a clicked location, a competition and catchment analysis that suggests lower-competition spots, a cost-to-open and break-even estimator, and a side-by-side comparison of up to three locations. The tools label measured values (counts, distances and median prices) separately from modelled estimates (rent, revenue and payback).

A benchmark of 39 spatial questions with hand-written gold queries shows that the engine returns exactly the gold answer for 74.4% of the questions (geometry F1 88.4%) and that every generated query ran. Most of the failures come from a row cap and the strictness of exact-match scoring. A direct-LLM baseline that names places and coordinates without a database returned names that exist in the database only 19.5% of the time. A sensitivity study of the earlier weighted-sum ranking showed that it is stable under small changes to one weight but not under a change of weighting scheme, which is why the new tools report measured quantities. The limitations are stated openly: a small single-run benchmark, no real rental data, heuristic demand and revenue models, and listing locations that are accurate only to the region. The work shows that having an LLM write a query that is executed on real data gives answers that are both useful and verifiable.

Keywords: natural-language interface, geospatial query, text-to-SQL, PostGIS, large language models, site selection

CO-PO MAPPING

CO-PO Mapping:

Map COs with POs, POs satisfied by respective COs are mentioned in the above table.
In the table given below, you have to map COs with POs in either of the three levels. Write levels against every COs. Write "-" if CO not mapping with PO.
"1": For less mapping "2": Moderate mapping "3": Strong mapping
(PSO1, PSO2, and PSO3 mapping with COs based on the Project development task. PSO2 will be mapped only if your project design task includes VLSI Design technology)

COs	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	3	3	2	3	2	1	-	-	1	-	2	-	2	2	-
CO2	2	-	3	-	3	-	-	-	-	1	2	-	3	2	-
CO3	-	2	2	-	-	3	2	-	-	-	1	-	1	-	-
CO4	-	-	-	-	1	-	-	2	3	-	2	-	-	-	-
CO5	-	-	-	-	-	-	3	-	2	-	1	-	-	-	-
CO6	-	-	-	-	-	-	-	3	1	3	2	-	-	-	-

CO/PO mapping	Proof required
CO1	Literature review chapter, detailed problem statement based on research knowledge, discussion on results
CO2	Hardware and software used for project
CO3	Include a chapter on the Introduction and Context of the project (Context related to **public health, safety, societal, environmental issues**, and sustainable development)
CO4	Project Report writing and presentation
CO5	Plagiarism check certificate, proper citations in the report, acknowledgment of figures, data, observations in rubrics
CO6	Timely completion- marks awarded in rubrics, Budget documentation- Logbook maintenance

Explanation of PO satisfied by LY Project

PO1 Engineering Knowledge (EK)	Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2 Problem Analysis (PA)	Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3 Design/Development of Solutions (DS)	Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4 Conduct Investigations of Complex Problems (IP)	Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5 Engineering Tool Usage (ETU)	Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6 The Engineer and The World (EW)	Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7 Ethics (Eth)	Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8 Individual and Collaborative Team work (ICTw)	Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9 Communication (Cm)	Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
PO10 Project Management and Finance (PM)	Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11 Life-long Learning (LL)	Recognize the need for, and have the preparation and ability for: i) Independent and life-long learning ii) Adaptability to new and emerging technologies and iii) Critical thinking in the broadest context of technological change. (WK8)

PROGRAM SPECIFIC OUTCOMES (PSOs)

Students of the program will:

PSO I	Exhibit proficiency in the field of **Artificial Intelligence** by **developing intelligent systems**
PSO II	Be accomplished in the field of **data analytics, statistical programming languages and Machine learning**

CONTENTS

Chapter No.	Title	Page No.
	List of Abbreviations	iii
	List of Symbols	iv
	List of Figures	vi
	List of Tables	vii
1	Introduction	1
1.1	Background	1
1.2	Motivation	1
1.3	Societal and Environmental Context	2
1.4	Problem Statement	3
1.5	Objectives	3
1.6	Scope of the Work	4
1.7	Contributions	4
1.8	Organisation of the Report	5
2	Literature Survey	6
2.1	Geospatial Analysis Tools and Web Mapping	6
2.2	Natural-Language Interfaces to Databases	6
2.3	Large Language Models for Geospatial Tasks	7
2.4	Spatial Decision Support and Multi-Criteria Analysis	7
2.5	Open Geospatial Data and Tools	8
2.6	Summary and Research Gap	8
3	Proposed System	10
3.1	System Overview	10
3.2	Requirements	11
3.3	Methodology of the Query Engine	11
3.4	Database Design	12
3.5	Query Engine Design	14
3.6	Decision Support Models	15
3.6.1	Site evaluation	16
3.6.2	Competition and catchment analysis	16
3.6.3	Cost-to-open and break-even	17
3.6.4	Location comparison	18
3.6.5	Weighted ranking of areas (legacy dashboard)	18
3.7	Data Sources and ETL Design	19

Chapter No.	Title	Page No.
3.8	User Interface Design	19
3.9	Security and Safety Design	20
4	Implementation	21
4.1	Hardware Requirements	21
4.2	Software Requirements	21
4.3	Implementation of the Data Layer	22
4.4	Implementation of the Backend	23
4.5	Implementation of the Web Interface	24
4.6	Running the System	25
4.7	Verification	25
5	Results and Discussion	27
5.1	Implemented System	27
5.2	Qualitative Demonstration	28
5.3	Benchmark of the Query Engine	32
5.3.1	Method	32
5.3.2	Accuracy of the engine	32
5.3.3	The direct-LLM baseline	33
5.3.4	Latency	34
5.4	Sensitivity of the Weighted Ranking	34
5.5	Evaluation Status	36
5.6	Discussion	36
6	Conclusion and Future Scope	38
6.1	Conclusion	38
6.2	Limitations	39
6.3	Future Scope	40
	Bibliography	41
A1	Schema Description Given to the SQL Agent	43
A2	Database Definition (Excerpt)	45
A3	Benchmark Questions	47
	Publications and Certificates	49

LIST OF ABBREVIATIONS

AHP – Analytic Hierarchy Process

API – Application Programming Interface

BHK – Bedroom, Hall and Kitchen

CRITIC – Criteria Importance Through Intercriteria Correlation

CSV – Comma-Separated Values

EPSG – European Petroleum Survey Group (coordinate reference system codes)

ETL – Extract, Transform and Load

GiST – Generalized Search Tree

GIS – Geographic Information System

JSON – JavaScript Object Notation

LLM – Large Language Model

MCDA – Multi-Criteria Decision Analysis

MVT – Mapbox Vector Tile

NL – Natural Language

OSM – OpenStreetMap

POC – Proof of Concept

POI – Point of Interest

REST – Representational State Transfer

ROC – Rank-Order Centroid

SQL – Structured Query Language

TOPSIS – Technique for Order of Preference by Similarity to Ideal Solution

UI – User Interface

LIST OF SYMBOLS

β – Benchmark rate of customers per day per 100 sq ft for a business type

ρ – Competitor density in competitors per km^2

τ – Kendall's rank correlation coefficient

a – Number of amenities within 1 km of a location

b – Transit indicator, 1 if a metro or rail stop lies within 800 m and 0 otherwise

c – Cost of goods as a share of revenue

c_k – Number of points of interest or transit stops in group k inside the radius

f – Footfall multiplier

g – Competition factor

m – Customer multiplier of a revenue scenario (0.7, 1.0 or 1.3)

m_{ik} – Score of area i on criterion k (0 to 10)

m'_{ik} – Score of area i on criterion k after inverting criteria where lower is better

n – Number of competitors inside the radius or within 1 km, as stated in the text

n_{500} – Number of competitors within 500 m of a candidate spot

p_{med} – Median sale price per sq ft of nearby listings in rupees

r – Radius of the analysis in km

t – Average ticket in rupees

w_k – Weight of criterion or group k

A – Store area in sq ft

C_k – Saturation cap of group k

D – Catchment demand index (0 to 100)

F – Fixed monthly costs in rupees

K – Number of criteria

N – Customers per day

O – Opportunity score of a candidate spot

P – Monthly profit in rupees

R – Monthly rent in rupees

S_i – Weighted score of area i

T – Payback period in months

U – One-time setup cost in rupees

V – Monthly revenue in rupees

V^* – Break-even monthly revenue in rupees

LIST OF FIGURES

Figure No	Title	Page No.
3.1	Architecture of the system	10
3.2	Pipeline of the natural-language query engine	11
5.1	Competition tab for a cafe at Vikhroli (B), 1.5 km radius: competitors in red, suggested spots in green, with the catchment demand index and the nearest competitors in the side panel.	28
5.2	Comparison of Kanjurmarg (A) and Vikhroli (B) for a standard-tier cafe. The site rows are measured within 1.5 km. Rent, start-up cost, revenue, profit and payback are modelled estimates from a single run.	29
5.3	Cost-to-open report for a cafe at Vikhroli: headline figures, one-time costs and the start of the monthly costs.	30
5.4	Cost-to-open report, continued: capital needed to start, modelled revenue scenarios and the assumptions used.	31
5.5	Exact-match accuracy by question category. The first-attempt-only configuration is identical to the full engine because no repair was needed.	33
5.6	Grounding of the direct-LLM baseline (three left bars) next to the engine's geometry F1 against the gold answer (right bar). The metrics differ: the baseline bars show how much of its output is real, the engine bar its agreement with the gold layer.	34
5.7	Rank agreement (Kendall's τ) of each weighting scheme with the hand-picked weights, and the overlap of the top-five areas.	35
5.8	One-at-a-time sensitivity: Kendall's τ against the hand-picked ranking when one criterion's weight is changed by up to 30%, the others being renormalised.	35

LIST OF TABLES

Table No.	Title	Page No.
2.1	Related work and how each source is used in this project	8
3.1	Schemas and tables of the spatial database	13
3.2	Revenue benchmarks for six of the business types supported (modelled assumptions)	18
4.1	Software used in the project	22
4.2	Endpoints of the backend	23
5.1	Mumbai data loaded in the current instance	27
5.2	Benchmark results on 39 questions, single run	32
5.3	What has and has not been measured	36
A3.1	Benchmark questions and the result of the full engine	47

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Almost every decision about land and buildings starts with a question about a place. Where should a clinic be built so that people can reach it? Which neighbourhood is safe and well connected for a family? Where can a new cafe open without being drowned out by competitors? Answering questions like these needs spatial data (roads, transit stops, buildings, businesses, property prices) and a way to combine it.

Geographic Information Systems (GIS) are the traditional tools for this work. Desktop systems such as ArcGIS and QGIS offer buffering, proximity analysis and spatial queries, but they assume that the user understands spatial data formats, coordinate systems and query languages such as SQL. A business owner who has a location question but no GIS training is therefore left outside the process. Web mapping platforms such as Felt [2] have made it much easier to build, share and discuss maps, yet the user still has to turn a real-world question into manual operations on layers and filters.

Large language models (LLMs) offer another way in: the user simply types the question in plain English. Recent work shows that LLMs can write executable queries for structured data and can even drive geospatial software [3], [4]. The risk is how the model is used. If the model is asked to answer the question itself, it may invent place names and coordinates, and nothing in its answer can be checked against data.

1.2 MOTIVATION

Our first proof of concept (POC) took exactly this direct approach, and it exposed three weaknesses. The model invented area names and coordinates. Areas were ranked by a fixed, hand-weighted formula over about 40 areas whose 1 to 10 metrics were synthetic. Map markers were placed with random coordinate jitter on Leaflet raster maps, so no real

spatial computation took place. The assistant answered with familiar area names, but none of its output could be verified.

This motivated the central idea of the project: replace "the LLM hallucinates the answer" with "the LLM writes a structured spatial query that is executed on real data". The answer then comes from the database, it is reproducible, and the SQL that produced it can be shown to the user as evidence.

A second motivation is practical. A person deciding where to open a cafe must weigh nearby property prices, transit access, safety, competition and start-up cost. No single easy tool combines these, so the work is spread over maps, property listings and reports and can take weeks of manual survey. We wanted one system in which a question can be asked in plain English, the evidence is shown on a map, and the follow-up decision tools (site report, competition analysis, cost estimate, side-by-side comparison) use the same data.

1.3 SOCIETAL AND ENVIRONMENTAL CONTEXT

The project works with data that describes how people live in a city, so its context goes beyond commerce. This section links the data layers of the system to public health, safety, society, the environment and sustainable development.

- **Public health.** The database holds hospitals, clinics, doctors and pharmacies from OpenStreetMap. Questions such as "hospitals within 1 km of a metro station" show how access to healthcare relative to public transport can be examined. Such answers may help an individual find a clinic, and the same query pattern can support a planner's review of access.
- **Safety.** The system includes an area-level safety score and risk level for neighbourhoods. A location report places this score next to price, transit and amenities, so safety is considered as part of the decision rather than discovered afterwards. The score is area-level only and describes an area, not a street or a person.
- **Society and economy.** Small businesses often decide on a location with little data. A tool that makes demand, competition and cost visible in one place lowers the barrier for owners without analysts. The database, tile server and map libraries are open source and the base data is open; the only external service is the hosted LLM that is called through an API.

- **Environment and sustainable development.** The transit layer (metro, rail, bus) and the amenity counts let a user see how well a place is served on foot and by public transport. Choosing locations that are close to public transport and daily amenities can reduce the need for long trips, which is consistent with the aims of sustainable and inclusive cities (United Nations Sustainable Development Goal 11). The system does not measure environmental outcomes itself; it makes the relevant spatial facts easy to see.
- **Ethics and responsible use.** The system uses open and aggregated data and does not collect personal data. Its cost and revenue figures are modelled estimates and are labelled as such, and the report states that they are planning aids and not financial advice. Every answer carries its SQL, so a reader can check how a result was obtained.

1.4 PROBLEM STATEMENT

Geospatial analysis is essential for business planning, but existing GIS tools demand knowledge of spatial data and query languages and mostly leave users to interpret patterns by hand. A person deciding where to open a cafe must weigh prices, transit, safety, competition and start-up cost, and no single easy tool combines these. LLM interfaces lower the barrier, but when the model supplies the answer directly, place names and coordinates can be invented and nothing can be verified. A system is needed that accepts natural language, computes answers on real spatial data, shows how each answer was obtained, and supports practical location decisions.

1.5 OBJECTIVES

The primary objective is to build a natural-language geospatial query and decision-support system whose answers are computed from real data. The specific objectives are:

1. **To provide a natural-language interface** that lets users ask spatial questions in plain English without SQL or GIS knowledge.
2. **To translate questions into PostGIS queries** using an LLM grounded in a documented schema, and to return the result as a map layer, an explanation and the SQL.
3. **To build a spatial data foundation** on PostgreSQL and PostGIS from OpenStreetMap, real-estate listings and crime data, with an ETL pipeline that generalises to other cities.

4. **To serve and render layers efficiently** using vector tiles, with a toggleable layer panel and click-to-inspect interaction.
5. **To support location decisions** through a site evaluation report, competition and catchment analysis, a cost-to-open and break-even estimator, and a side-by-side comparison of candidate locations.
6. **To keep results explainable** by generating plain-English explanations and exposing the executed SQL.
7. **To remain cost-effective** by using open-source tools and open data.

1.6 SCOPE OF THE WORK

The deployed instance is loaded with data for Mumbai: administrative boundaries, roads, transit stops, businesses and points of interest from OpenStreetMap, a 76,000-row house-price dataset, and an area-level crime and safety dataset. Every table carries a city identifier and the loading pipeline is driven by a city polygon, so the design supports other cities, but only Mumbai has been loaded and tested.

The user interface is a desktop web application. Questions are asked in English. The query engine returns one read-only query per question; it does not modify data. The decision tools are intended as planning aids: the measured parts (counts, distances, median prices) come from the database, and the modelled parts (rent, revenue, payback) are labelled as estimates. Real-time data, travel-time catchments and saved user sessions are outside the current scope (Chapter 6 lists them as future work).

1.7 CONTRIBUTIONS

The main contributions of this work are:

1. A natural-language to PostGIS query engine that grounds the LLM in a documented schema and returns a GeoJSON layer, an explanation and the executed SQL.
2. A real-data foundation: an ETL pipeline that loads OpenStreetMap, real-estate listings and crime data into one spatial database, designed to work for any city.
3. Decision tools (site evaluation, competition analysis, cost-to-open, location comparison) built on the same data.

4. An honest account of what is and is not yet validated, with a first benchmark of query accuracy on 39 spatial questions and the evaluation plan for the missing pieces.

1.8 ORGANISATION OF THE REPORT

Chapter 2 reviews related work on GIS tools, natural-language interfaces to databases, LLMs for geospatial tasks and spatial decision support. Chapter 3 describes the proposed system: its architecture, database design, query engine and decision models. Chapter 4 gives the implementation details, including the hardware and software used. Chapter 5 presents the results: the implemented system, a qualitative demonstration, the query-accuracy benchmark and a sensitivity study of the legacy ranking. Chapter 6 concludes the report and lists limitations and future scope.

CHAPTER 2
LITERATURE SURVEY

This chapter reviews the work that this project builds on. It looks at geospatial tools and web mapping, at natural-language interfaces to databases, at the recent use of LLMs for geospatial tasks, at spatial decision support, and at the open data and software that make the system possible. It ends with a summary of what each source contributes and the gap that this work addresses.

2.1 GEOSPATIAL ANALYSIS TOOLS AND WEB MAPPING

Traditional GIS platforms such as ArcGIS and QGIS provide buffering, proximity analysis and spatial querying, but they assume knowledge of spatial concepts, data formats and query languages. Cloud platforms such as Felt and Mapbox improve collaboration and visualization [2], yet users still drive them through layers, filters and map controls. Felt has also added AI features to its platform [2]. The layered, shareable web map is therefore a useful interaction model, and our workspace follows it: the user sees toggleable layers with legends on a vector-tile map.

2.2 NATURAL-LANGUAGE INTERFACES TO DATABASES

A natural-language interface to a database maps a question to an executable query. Spider [5] is a large human-labelled dataset for complex, cross-domain semantic parsing and text-to-SQL, designed so that systems are tested on databases they have not seen. BIRD [6] is a larger benchmark, with 12,751 question and SQL pairs over 95 databases of realistic size (33.4 GB in total) from 37 professional domains. Together these benchmarks have driven progress on general text-to-SQL, and LLM-based systems now dominate the leaderboards.

Spatial questions add difficulty. The model has to pick the right geometry types, apply spatial predicates and distance functions correctly, and respect coordinate reference systems. Early geospatial interfaces used rule-based mapping from keywords such as

"near" and "within" to spatial operations. These work for simple queries but are brittle for ambiguous or compositional ones.

2.3 LARGE LANGUAGE MODELS FOR GEOSPATIAL TASKS

Spatial text-to-SQL with multiple agents. Kazazi et al. [4] present a multi-agent framework for spatial text-to-SQL. A notable component is an execution-based review stage, in which the generated query is run and the outcome is used to correct it. They report that the stage raises accuracy on their SpatialQueryQA benchmark from 76.7% to 87.7%, an improvement of about 11 points. That figure is a result from their work, not from ours. We follow the same design idea in a simplified form: our engine runs each generated query and uses a database error or an empty result to ask the model for a corrected query.

Geospatial analysis for non-experts. ChatGeoAI [3] converts natural-language requests into executable geospatial operations. It is built on a Llama 2 model and produces PyQGIS scripts for QGIS, so that users without programming skills can run analyses. Our system has the same goal of serving non-experts, but it targets a spatial database and returns a map layer together with the query that produced it.

Geospatial knowledge in LLMs. GeoLLM [7] shows that LLMs hold geospatial knowledge and that combining the model with information from OpenStreetMap can model quantities for which ground truth is scarce. We treat this as relevant to estimating metrics, such as rent, where no direct data exists. In our system such modelled values are always labelled as estimates.

2.4 SPATIAL DECISION SUPPORT AND MULTI-CRITERIA ANALYSIS

Spatial decision support systems extend GIS with multi-criteria decision analysis (MCDA) for tasks such as site selection [1]. They rank candidate locations by combining normalised criteria with weights, but typically need expert configuration and expert interpretation of the result. Our legacy dashboard used a weighted-sum model of this kind. Chapter 5 studies how sensitive that ranking is to the choice of weights, and the decision tools of the new system report measured quantities instead of a single weighted score.

2.5 OPEN GEOSPATIAL DATA AND TOOLS

OpenStreetMap (OSM) provides free, globally available data on roads, amenities and land use [8]. It is produced by a community of volunteers in the same peer-production manner as Wikipedia, and its completeness varies from place to place. OSMnx [9] is a Python library that makes it easy to download and analyse OSM data, including extracts bounded by a city polygon. We use it to load any city into our database.

PostGIS adds spatial types, GiST indexing and spatial functions to the PostgreSQL database, which makes it a suitable target for generated queries: a single SQL statement can express a buffer, a distance filter and a join between layers. Martin serves Mapbox Vector Tiles directly from PostGIS, so the map is rendered from the same data that the queries use.

2.6 SUMMARY AND RESEARCH GAP

Source	**What it contributes**	**How this project uses it**
Spider [5] BIRD [6]	Benchmarks for general text-to-SQL	Design reference for our benchmark of spatial questions
Kazazi et al. [4]	Multi-agent spatial text-to-SQL with an execution-based review stage	Intent check, schema grounding, SQL generation and a bounded repair loop
Chat-GeoAI [3]	Natural language to executable geospatial operations for non-experts	Same goal; we generate SQL and show it with the map layer
GeoLLM [7]	LLM knowledge combined with OSM features	Relevant to modelled values such as rent, which we label as estimates
Malczewski [1]	Multi-criteria decision analysis with GIS	Weighted-sum ranking in the legacy dashboard and its sensitivity study
OSM [8] OSMnx [9]	Open map data and tools to acquire it	Source of roads, transit and businesses; polygon-based city ingest
Felt [2]	Layered, shareable web mapping with AI features	Interaction model for the layered workspace

Table 2.1: Related work and how each source is used in this project

Existing systems tend to cover either visualization, query processing or decision support. A system that lets the model answer directly is easy to build, but its output cannot be checked. The gap this project addresses is the combination of the three on one open-data stack, with the emphasis on verifiable query results: a natural-language question is translated into a validated spatial query, executed on real data, shown on a map with the SQL that produced it, and followed by decision tools that use the same data (Table 2.1).

CHAPTER 3
PROPOSED SYSTEM

This chapter describes the system that was built: its overall architecture, the requirements it meets, the pipeline that turns a question into a map layer, the database, the query engine and the decision models. Everything described here is implemented; Chapter 4 gives the implementation details and Chapter 5 the results.

3.1 SYSTEM OVERVIEW

The system has three layers (Figure 3.1). The **presentation layer** is a web application built with React and MapLibre GL. It shows a full-screen vector-tile map with a panel of toggleable layers, a natural-language query bar, and panels for evaluating, comparing and costing locations. The **application layer** is a FastAPI backend. It receives questions and location requests, calls the LLM through a single client module, queries the database and returns JSON and GeoJSON. The **data layer** is a PostgreSQL database with the PostGIS extension, which holds all spatial data, and the Martin tile server, which publishes the same tables as vector tiles.

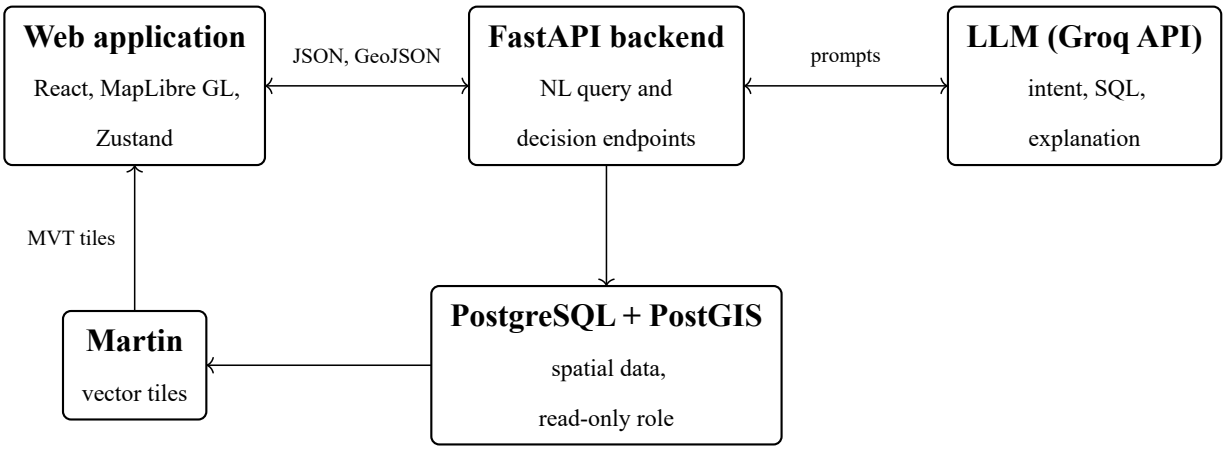


Figure 3.1: Architecture of the system

A question travels through the system as follows. The browser sends it to the backend, where an LLM pipeline checks the intent of the question, grounds it in a description of the schema and generates a PostGIS query. The query is validated and executed on the database under a read-only role. If the database returns an error or no rows, a correction

loop asks the model for a better query. The resulting rows are converted into a GeoJSON answer layer, which the map draws above the base layers together with an explanation and the SQL. The decision tools reuse the same database through their own endpoints.

3.2 REQUIREMENTS

Functional requirements. The system shall:

1. accept a spatial question in plain English and return an answer layer on the map, a short explanation and the SQL that was executed;
2. ask a short follow-up question when the target of a question cannot be determined;
3. display the base layers (boundaries, roads, transit stops, businesses, real-estate listings, safety score) with a legend and an opacity control for each;
4. produce a site report for a clicked location, a competition and catchment analysis for a business type, a cost-to-open and break-even estimate, and a side-by-side comparison of up to three locations;
5. load a new city from OpenStreetMap and the other sources without changes to the code.

Non-functional requirements. Answers must be verifiable (no place or number is invented by the model); generated SQL must never be able to modify data; the application must start and report errors cleanly when the database or the LLM is unavailable; and the whole stack must be reproducible on a standard computer with open-source software.

3.3 METHODOLOGY OF THE QUERY ENGINE

The query engine is a pipeline of focused steps (Figure 3.2). Each LLM call has a narrow job and a fixed output format.

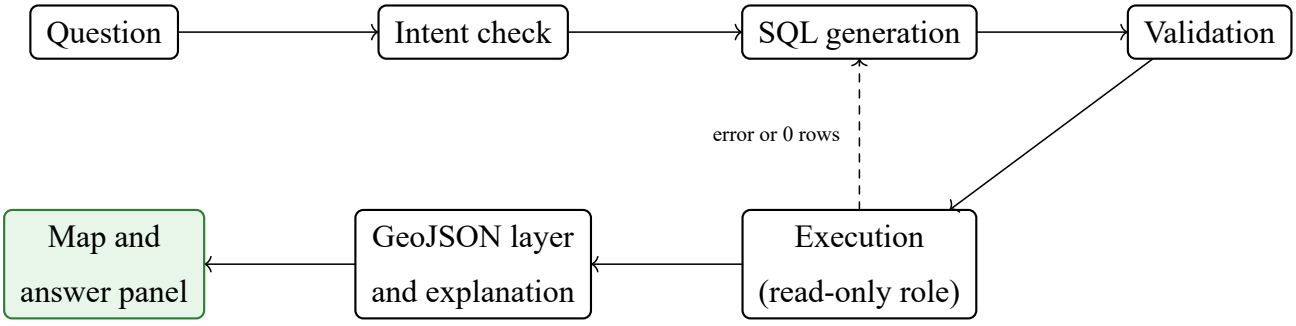


Figure 3.2: Pipeline of the natural-language query engine

1. **User input.** The user types a spatial question, either exploratory (“hospitals within 1 km of a metro station”) or decision-oriented.
2. **Intent check.** A first LLM call decides whether the question is a clear spatial query and labels its task type (filter, proximity, ranking, aggregation, site selection or other). If the target is genuinely unknowable, for example “best place?”, the system returns a short follow-up question instead of running a query.
3. **Schema grounding and SQL generation.** The second call receives a compact description of the queryable schema and writes one PostGIS query. Because the model can only reference tables and columns that exist, it cannot invent places.
4. **Validation and execution.** The query is checked and then executed on the database under a read-only role.
5. **Repair.** If the query fails or returns no rows, the error or a hint to loosen the query is sent back to the model, within fixed limits.
6. **Result generation.** The rows are converted into a GeoJSON answer layer, and a third call writes a short explanation based on the rows that were actually returned.

The design follows the multi-agent spatial text-to-SQL approach of Kazazi et al. [4] in a simplified form: it uses the intent, grounding, generation and execution-review stages, but keeps the stages in one service and uses a fixed schema description instead of retrieval.

3.4 DATABASE DESIGN

All spatial data lives in one PostgreSQL 16 database with the PostGIS 3.4 extension. The data is divided into ten schemas, one per layer group (Table 3.1). All geometry is stored in EPSG:4326, and metric operations cast to the `geography` type at query time. Every feature table has a `city_id` column that refers to `core.city`, so one database can hold several cities and every query is scoped to one of them.

Schema	Table	Main columns
`core`	`city`	name, country, centroid, bounding box, status
`admin`	`boundary`	name, admin level, kind (city, ward), multipolygon
`roads`	`segment`	name, highway class, maximum speed, one-way flag, line
`transport`	`stop`, `line`	name, mode (bus, metro, rail, tram), point or line
`poi`	`place`	name, category, tags (JSON), point
`realestate`	`listing`	locality, BHK, type, area in sqft, price, price unit, status, age, point
`demo`	`cell`	population, density, median age, income index, polygon
`crime`	`area`, `incident_agg`	name, zone, safety score (0 to 100), risk level, breakdown; aggregated incidents
`env`	`measurement`	kind (air quality, flood risk), value, timestamp, geometry
`meta`	`schema_catalog`	table and column names, data type, geometry flag, description, vector embedding

Table 3.1: Schemas and tables of the spatial database

The geometry columns of the feature tables have GiST spatial indexes and their `city_id` columns have B-tree indexes; the category column of `poi.place` is indexed for category filters and the locality column of `realestate.listing` has a trigram index for fuzzy name matching. Two points in the design deserve mention:

- **Neighbourhood gazetteer.** OpenStreetMap ward names are administrative codes such as “K/W Ward” and do not contain neighbourhood names like “Bandra”. The `crime.area` table has about 40 named neighbourhood points, so the engine uses it to resolve a colloquial area name to a location.
- **Schema catalog.** The `meta.schema_catalog` table documents every table and column (34 rows in the current instance) and has a vector column for embedding-based retrieval. The query engine does not read this table at query time; instead the same description is written into the prompt (next section).

Two database roles are used. The owner role is used by the ETL pipeline and by the decision endpoints. The `geo_readonly` role, which has `SELECT` rights on the data schemas and nothing else, is the only role that executes LLM-generated SQL.

3.5 QUERY ENGINE DESIGN

Intent agent. The first prompt describes the available data and asks for strict JSON with four fields: whether the question is clear, its task type, a one-sentence confirmation or follow-up question, and the entities it mentions. The prompt tells the model to lean towards "clear" and to ask for clarification only when the target is genuinely unknowable.

SQL agent. The second prompt contains a compact description of the schema and a set of rules. The description lists each table with its geometry type and the allowed values of its key categorical columns, for example the transit `mode` values and the `poi.place` categories. The rules are:

- filter every table by `city_id`;
- businesses and amenities live only in `poi.place`, and transit stops are never amenities;
- resolve neighbourhood names through the `crime.area` gazetteer with an `ILIKE` match and a distance buffer;
- return exactly one read-only `SELECT` statement that exposes a geometry column named `geom`;
- use `geography` casts for distances in metres, match names with `ILIKE`, and use `ORDER BY` with a `LIMIT` of at most 500 when ranking;
- never reference a table or column that is not listed.

Validation. The text returned by the model is cleaned of any markdown fence and then checked by a short function (Listing 3.1). It must be a single statement that starts with `SELECT` or `WITH`, and it must not contain a data-modifying or definition keyword. The check is deliberately simple; the read-only role is the second, stronger barrier.

```
_FORBIDDEN = re.compile(
    r"\b(insert|update|delete|drop|alter|create|truncate|grant|"
    r"revoke|copy|vacuum|call|merge|do)\b", re.IGNORECASE)

def validate_select(sql: str) -> str:
    s = _strip_sql(sql).rstrip(";").strip()
    if ";" in s:
        raise ValueError("multiple statements are not allowed")
    low = s.lower()
    if not (low.startswith("select") or low.startswith("with")):
        raise ValueError("query must be a SELECT")
    if _FORBIDDEN.search(s):
        raise ValueError("query contains a forbidden keyword")
    return s
```

Listing 3.1: Validation of generated SQL before execution

Execution. The validated query is wrapped so that the database itself builds a GeoJSON `FeatureCollection` with `ST_AsGeoJSON`, limited to 5,000 features. It runs as `geo_readonly` with a 15 second statement timeout.

Repair. If validation or execution fails, the first line of the error text (at most 400 characters) is sent back to the model with a request for a corrected query. At most two such repairs are made. If a valid query returns zero rows, the model is asked once to loosen the query, for example by using `ILIKE` instead of equality, resolving area names through the gazetteer, or widening the buffer. In total there are at most four attempts. The best successful attempt is kept, preferring one that returned rows, and every attempt is returned to the client so that the interface can show what was tried.

Explanation agent. The third prompt receives the question, the number of rows and the properties of up to 15 returned features, and writes two to four sentences describing what was found. If the explanation call fails, the engine returns a plain count instead.

3.6 DECISION SUPPORT MODELS

The decision tools are built on the same database and are exposed as separate endpoints. They separate what is **measured** (counts, distances and median prices computed from the data) from what is **modelled** (rent, revenue and payback), and the interface labels the second kind as estimates.

3.6.1 Site evaluation

For a clicked point and a radius, the report gives the median price per square foot of nearby listings with a typical price band, a comparison with the city median, a breakdown by size band, the nearest metro, rail and bus stops, amenity counts in five groups (healthcare, education, food, banking and shopping), a safety score with a risk level taken from the nearest `crime.area` point, and the best available locality name. Everything in this report is measured.

3.6.2 Competition and catchment analysis

Competitors are the points in `poi.place` whose category matches the business type inside the chosen radius r (for a cafe, the category `cafe`). The report gives their count n, the nearest one and the density

$$\rho = \frac{n}{\pi r^2} \tag{3.1}$$

in competitors per square kilometre. A saturation label follows fixed bands: below 5 is low, 5 to 15 moderate, 15 to 30 high and above 30 saturated.

The catchment demand index counts the points of interest in five groups and the transit stops inside the radius. Each count c_k is capped at a saturation value C_k (40 for healthcare, 25 for education, 40 for food, 15 for banking, 30 for shopping and 12 for transit stops), so that very dense areas do not dominate, and the capped counts are combined with business-specific weights w_k:

$$D = \operatorname{round}\left(100 \frac{\sum_k w_k \min\left(\frac{c_k}{C_k}, 1\right)}{\sum_k w_k} \right) \tag{3.2}$$

For a cafe the weights are 0.25 for education, 0.20 for shopping, 0.10 each for banking and food, 0.05 for healthcare and 0.30 for transit; other business types have their own weights, and a default set is used for unknown types.

To suggest gaps, a 5×5 grid of candidate spots is laid over the radius. For each spot the same index D is computed within 500 m, and the opportunity score is

$$O = \frac{D}{1 + n_{500}} \tag{3.3}$$

where n_{500} is the number of competitors within 500 m of the spot. Up to three spots with $D \geq 15$ and an opportunity score at least 15% higher than that of the evaluated spot are suggested. The saturation bands and the demand weights are heuristics set by judgement; they have not been calibrated against business outcomes.

3.6.3 Cost-to-open and break-even

The estimator takes the business type, a store area A in square feet and a tier (economy, standard or premium). Because the dataset has no rental listings, the monthly rent is derived from sale prices:

$$R = p_{\text{med}} \cdot 0.005 \cdot 1.15 \cdot A \tag{3.4}$$

where p_{med} is the median sale price per square foot of listings within 2.5 km, 0.005 is a monthly rent-to-value ratio and 1.15 is a commercial premium. If no listing is found nearby, a city default of 120 rupees per square foot per month is used. The security deposit is six months of rent, and the working capital is three months of the total monthly cost. The remaining line items (interior, furniture, equipment, initial stock, licences, branding, utilities, marketing and staff) are produced by the LLM from a prompt that fixes the rent; if the LLM is unavailable a heuristic fallback based on floor area is used, so the tool never fails.

The revenue model estimates the customers per day N as

$$N = \frac{A}{100} \cdot \beta \cdot f \cdot g \tag{3.5}$$

where β is a benchmark rate of customers per day per 100 square feet for the business type (Table 3.2), f is a footfall multiplier and g a competition factor:

$$f = 0.6 + 0.8 \min\left(1, \frac{a}{120}\right) + 0.1b, \quad g = \max\left(0.6, \frac{1}{1 + 0.04n}\right) \tag{3.6}$$

Here a is the number of amenities within 1 km, b is 1 if a metro or rail stop lies within 800 m and 0 otherwise, and n is the number of competitors of the same type within 1 km. The footfall multiplier therefore ranges from 0.6 in a quiet pocket to 1.5 in a dense,

transit-served one. Monthly revenue for a scenario with customer multiplier m (0.7 for conservative, 1.0 for base and 1.3 for optimistic) is

$$V = mNt \cdot 30 \tag{3.7}$$

where t is the average ticket, scaled by 0.8, 1.0 or 1.3 for the economy, standard and premium tiers. With cost of goods c as a fixed share of revenue and fixed monthly costs F (the monthly total without the restock line, which c replaces), the profit, the payback in months and the break-even revenue are

$$P = V(1-c) - F, \quad T = \frac{U}{P}, \quad V^* = \frac{F}{1-c} \tag{3.8}$$

where U is the one-time setup cost. The per-type benchmarks are modelling assumptions set by the authors and are not measured values (Table 3.2).

Business type	**Customers per day per 100 sqft (β)**	**Average ticket t (rupees)**	**Cost of goods c**
Cafe	18	300	30%
Restaurant	9	650	33%
Pharmacy	30	450	72%
Clinic	5	700	10%
Grocery store	22	400	78%
Gym	1.5	1,800	5%

Table 3.2: Revenue benchmarks for six of the business types supported (modelled assumptions)

3.6.4 Location comparison

Up to three pinned spots (A, B and C) are shown side by side across price, access, safety, number of competitors, rent, start-up cost, revenue, profit and payback. The best value in each row is marked. The site rows are measured within 1.5 km; the business rows are the modelled estimates described above.

3.6.5 Weighted ranking of areas (legacy dashboard)

The original dashboard ranks areas with a standard weighted-sum multi-criteria model [1]. For area i with criteria m_{ik} (footfall, youth, access, rent and competition, each scored from 0 to 10) the score is

$$S_i = \sum_{k=1}^{K} w_k \, m'_{ik}, \quad \sum_{k=1}^{K} w_k = 1 \tag{3.9}$$

where $m'_{ik} = m_{ik}$ for benefit criteria (footfall, youth, access) and $m'_{ik} = 10 - m_{ik}$ for criteria where lower is better (rent and competition). The hand-picked weights are 0.30, 0.25, 0.20, 0.15 and 0.10 in that order. In the POC the metrics were synthetic, which is why the new decision tools do not use this ranking; Chapter 5 studies how sensitive it is to the choice of weights.

3.7 DATA SOURCES AND ETL DESIGN

The ETL pipeline loads one city at a time and each layer runs independently, so a failure in one layer does not stop the others. A city is marked ready when its core layers have loaded. The pipeline can be re-run for the same city: each layer first clears that city's rows and then inserts them again.

- **OpenStreetMap.** The city polygon is resolved first. Administrative boundaries, the road network (major road classes), transit stops and points of interest are then extracted inside that polygon with OSMnx [9] and stored with the city identifier.
- **Real estate.** A house-price dataset of about 76,000 rows is loaded. The data has no coordinates, so each distinct region (228 in the current instance) is geocoded once with Nominatim and the result is cached. Every listing is placed at the centre of its region plus a small deterministic offset of up to about 0.006 degrees (roughly 700 m), so that listings of the same region form a readable cloud instead of stacking on one point. Listing positions are therefore indicative of the region and are not property-level.
- **Crime and safety.** An area-level dataset gives each of about 40 Mumbai neighbourhoods a safety score from 0 to 100, a risk level and a breakdown by category. It is stored as points and doubles as the neighbourhood gazetteer.
- **Schema catalog.** A final step writes the schema description into `meta.schema_catalog`.

3.8 USER INTERFACE DESIGN

The main workspace is a full-screen vector-tile map. The layers are defined in one registry, so adding a layer is a single entry. The current layers, in their groups, are administrative

boundaries; the road network and transit stops (coloured by mode); businesses and points of interest (coloured by category); real-estate listings; and the area-level safety score (red for lower, amber for medium and green for higher safety). Each layer can be switched on and off and has an opacity control and a legend.

The query bar sits on the map. A question returns an answer layer drawn above the base layers, with the explanation and the generated SQL in a side panel. A button lets the user click any point on the map to open the evaluation panel, which has the site report, a **Competition** tab and a **Cost-to-open** tab. Competitors are drawn as red dots and suggested spots as numbered green markers. Locations can be pinned and compared in a comparison panel. Clicking any feature on the map shows its attributes.

3.9 SECURITY AND SAFETY DESIGN

Generated SQL is treated as untrusted. Four barriers apply, in this order: the prompt forbids anything but one `SELECT`; the validation function rejects multiple statements and definition or modification keywords; the query runs as the `geo_readonly` role, which can only read; and the statement timeout (15 seconds) and the feature cap (5,000) limit the cost of a bad query. The backend's own queries pass their values as bound parameters. The two checks in front of the database are simple and are not a full SQL parser, which is listed as a limitation in Chapter 6.

The backend is written so that it starts when the database or the LLM is unavailable. Endpoints then report the failure instead of crashing, and each LLM-backed feature has a deterministic fallback, as the cost estimator does. Secrets such as the API key are read from environment files and are not stored in the repository.

CHAPTER 4
IMPLEMENTATION

This chapter lists the hardware and software that the project uses and describes how each part of the design in Chapter 3 was implemented: the database and tile server, the backend, the data loading pipeline, the web interface, and the way the system is run and checked.

4.1 HARDWARE REQUIREMENTS

The system runs on a standard personal computer or laptop.

- **Processor:** Intel i5 or equivalent (a higher-end processor is recommended).
- **Memory:** at least 8 GB of RAM (16 GB recommended for large datasets).
- **Storage:** at least 20 to 50 GB of free space for the OpenStreetMap data and the database.
- **Network:** an internet connection for the LLM API and for geocoding.

4.2 SOFTWARE REQUIREMENTS

The stack is primarily open source (Table 4.1). The only external service is the hosted LLM.

Sr. No.	Software or tool	Use in the project
1	PostgreSQL 16 with PostGIS 3.4	Spatial database with GiST indexing; the `pgvector` and `pg_trgm` extensions are also installed
2	Martin 0.14	Serves Mapbox Vector Tiles directly from PostGIS
3	Python 3.12, FastAPI and Uvicorn	REST backend for the query engine and the decision tools
4	SQLAlchemy 2, asyncpg, GeoAlchemy2, Pydantic 2	Asynchronous database access and request validation
5	Groq API, model `openai/gpt-oss-120b`	LLM for intent checking, SQL generation, explanations and cost line items
6	React 19 and Vite 8	Web interface and its build tool
7	MapLibre GL 4	Vector-tile map rendering in the main workspace
8	Zustand 5, axios, React Router 7	Shared map state, HTTP calls and page routing
9	Leaflet	Map in the legacy dashboard only
10	OSMnx and GeoPandas	Polygon-based OpenStreetMap extraction and spatial data handling in the ETL
11	Nominatim	Geocoding of the real-estate regions
12	MongoDB	Chat history of the legacy dashboard only
13	Docker Compose	Runs the database and the tile server

Table 4.1: Software used in the project

4.3 IMPLEMENTATION OF THE DATA LAYER

Database container. The database runs in Docker. The image is built from the official `postgis/postgis:16-3.4` image, which bundles PostGIS 3.4 on PostgreSQL 16, with the `pgvector` package added. On the first start with an empty volume the container runs four initialisation scripts in order: the extensions (`postgis`, `postgis_raster`, `vector`, `pg_trgm`), the schema with all tables and indexes, the roles (including the read-only role), and a seed record for Mumbai. The database is published on host port 5433 so that it does not clash with a native PostgreSQL installation.

Vector tiles. Martin runs in a second container and connects to the same database. It is configured to publish every table of the data schemas automatically under the name

`schema.table`, which is also the identifier that the web interface uses for each layer. The tile server is available on host port 3001.

ETL pipeline. The loader is run from the command line for one city at a time:

```
python -m etl.run --city "Mumbai, India"
```

It resolves the city polygon and then loads the administrative boundaries, ward polygons, roads, transit stops and points of interest from OpenStreetMap, geocodes and loads the real-estate dataset, loads the crime and safety dataset, and finally writes the schema catalog. Each layer runs independently, so one failing layer does not abort the others, and individual layers can be skipped with an option. Because extraction is driven by the city polygon, the same command loads any other city.

4.4 IMPLEMENTATION OF THE BACKEND

The backend is a FastAPI application. Its entry point registers one router per feature, and each router is a thin module that defines the request model and calls a service, so that the logic can be tested without the web layer. Table 4.2 lists the current endpoints.

Endpoint	Purpose and main inputs	Service
`POST /nlquery`	Natural-language question and city; returns the GeoJSON answer layer, explanation, SQL and the list of attempts	`geo_engine`
`POST /site-eval`	Point and radius; returns the site report	`site_evaluator`
`POST /competitor-analysis`	Point, business type and radius; returns competitors, saturation, demand index and suggested spots	`competitor_analysis`
`POST /business-estimate`	Business type, point, area and tier; returns the cost and break-even estimate	`business_estimator`
`GET /health`	Reports whether the application can reach the database	

Table 4.2: Endpoints of the backend

The original dashboard additionally uses the earlier endpoints for analysis, generation, explanation, chat queries, location insight and property lookup. They are kept as the baseline and are not used by the new workspace.

Two database engines. The module that creates database connections offers two engines. The owner engine is used by the decision endpoints. The read-only engine, which logs in as `geo_readonly` and sets a 15 second statement timeout, is the only one the query engine may use, and it is only reached after the generated SQL has passed validation. Both engines are created lazily, so importing the application never fails when the database is down.

LLM client. All calls to the LLM go through a single module, which also contains a helper that parses the JSON replies of the model safely. Keeping one client makes the model easy to swap and means that prompts and parsing are in one place. Every feature that depends on the LLM has a fallback: the intent and SQL stages report a clear error, the explanation falls back to a row count, and the cost estimator falls back to a size-based heuristic.

Decision services. The three services described in Chapter 3 run their queries with bound parameters and compute the models in Python. The site report queries the nearby listings, the transit stops, the amenities and the nearest safety record. The competition service fetches the points of interest and transit stops around the point once and then computes the competitor list, the demand index and the 5×5 grid scan in memory. The cost estimator combines a median-price query, an LLM call and the revenue model; the revenue part is additive, so a failure there never prevents the cost estimate from being returned.

4.5 IMPLEMENTATION OF THE WEB INTERFACE

The interface is a single-page React application built with Vite. The main page, the **Studio**, contains the map and its panels:

- **Map canvas and layers.** MapLibre GL draws the base map and the vector-tile layers from Martin. The layers are described in one registry file, in which each entry gives the source, the paint rules (for example, transit stops coloured by mode and points of

interest coloured by category), the legend and whether the layer is visible at start. A layer panel reads the same registry to build the toggles and opacity sliders.

- **Query bar.** The bar sends the question to `/nlquery` and draws the returned GeoJSON as a new layer above the base layers. A side panel shows the explanation and the SQL, and shows a follow-up question when the engine asks for one.
- **Evaluation panel.** A button lets the user pick a point on the map. The panel then shows the site report, a **Competition** tab and a **Cost-to-open** tab, and the radius can be changed. Competitors are drawn as red dots and the suggested spots as numbered green markers.
- **Comparison panel.** Up to three evaluated locations can be pinned and compared side by side.

Shared map state is kept in one Zustand store instead of being passed through component properties, and every HTTP call is defined in one API module. The earlier pages of the proof of concept (home, dashboard, crime map and chat query) are kept in the application as the baseline.

4.6 RUNNING THE SYSTEM

The system is started in three steps: the database and tile server with `docker compose up -d`, the backend with Uvicorn on port 8000, and the frontend with the Vite development server on port 3000. A script for Windows starts all three together. Settings such as the database connection strings, the tile server address and the LLM API key are read from environment files, and a template file lists every setting. After the first start the ETL command above loads a city; the tile server is then restarted so that it picks up newly created tables. The `/health` endpoint shows whether the backend can reach the database.

4.7 VERIFICATION

The project has no automated unit-test suite. Verification was done in the following ways:

- **Benchmark script.** The query-accuracy benchmark in the evaluation folder has a lint mode that checks the question set without a database or an LLM, and a gold-validation mode that executes every hand-written gold query on the database without an LLM. The

full benchmark then compares the engine with its ablations and with the direct-LLM baseline (Chapter 5).

- **Health check and manual testing.** The backend has a `/health` endpoint that reports whether it can reach the database. The endpoints and the interface were exercised by hand against the loaded Mumbai data; the screenshots in Chapter 5 come from such runs.
- **Minimum checks for a change.** The project convention is that the Python modules are syntax-checked, that the frontend passes its build and lint commands, and that `/health` and a real request succeed before a change is accepted.

CHAPTER 5
RESULTS AND DISCUSSION

This chapter presents what the system does and how well it does it. It starts with the implemented system and the data it holds, shows the decision tools on a worked example, and then reports two studies: a benchmark of the query engine against hand-written gold queries, and a sensitivity study of the legacy weighted ranking. It ends with a discussion of what the results do and do not show.

5.1 IMPLEMENTED SYSTEM

The system is implemented end to end: the database and tile server, the query engine, the decision tools and the web workspace. Table 5.1 lists the Mumbai data that the current instance holds.

Dataset	**Size**
Administrative boundaries	48
Road segments	16.8 thousand
Transit stops	2.2 thousand
Businesses and points of interest	10.8 thousand
Real-estate listings (CSV rows)	76,000
Geocoded real-estate regions	228
Neighbourhoods with a safety score	40
Schema catalog entries	34

Table 5.1: Mumbai data loaded in the current instance

Compared with the proof of concept, the new workspace replaces invented coordinates with coordinates from the database, replaces random marker jitter with real geometries, replaces raster Leaflet maps with vector tiles, and replaces fixed synthetic metrics with metrics computed from nearby listings, transit and amenities. One exception should be noted: the real-estate listings have no coordinates of their own, so each is placed near the centre of its region (Chapter 3), and values computed from listings are therefore valid at the scale of a neighbourhood, not of a building.

5.2 QUALITATIVE DEMONSTRATION

A question such as “hospitals within 1 km of a metro station” is turned into a spatial SQL query, executed on the database and drawn as an answer layer, with the explanation and the SQL available in the side panel. Clicking a location opens a site report with price, access, amenity and safety information, and the Competition and Cost-to-open tabs extend it to a business decision.

Figure 5.1 shows this for a cafe at Vikhroli. The competition tab finds four cafes within 1.5 km, the nearest 1.3 km away, so the spot has no competitor within 1 km and a low saturation label. The catchment demand index is 78 out of 100. The tab also suggests two lower-competition spots (green markers), one 1.4 km away with demand 54 and one 675 m away with demand 35, each with no competitor within 500 m. The comparison panel (Figure 5.2) puts the Vikhroli spot (B) next to Kanjurmarg (A). The measured rows differ in ways a decision maker can use: the nearest metro or rail stop is 719 m from B and 929 m from A, B has 172 amenities within 1.5 km against 46 for A, and A has the higher safety score (82 against 74). Both sites have the same median price per square foot (about 19,000 rupees) and neither has a competitor within 1 km.

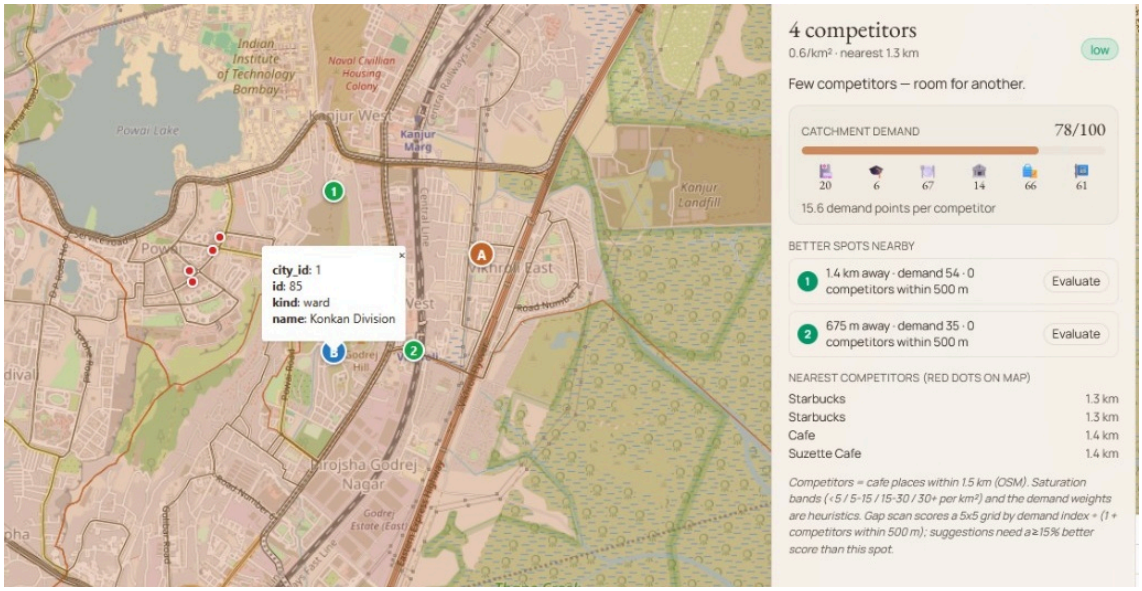


Figure 5.1: Competition tab for a cafe at Vikhroli (B), 1.5 km radius: competitors in red, suggested spots in green, with the catchment demand index and the nearest competitors in the side panel.

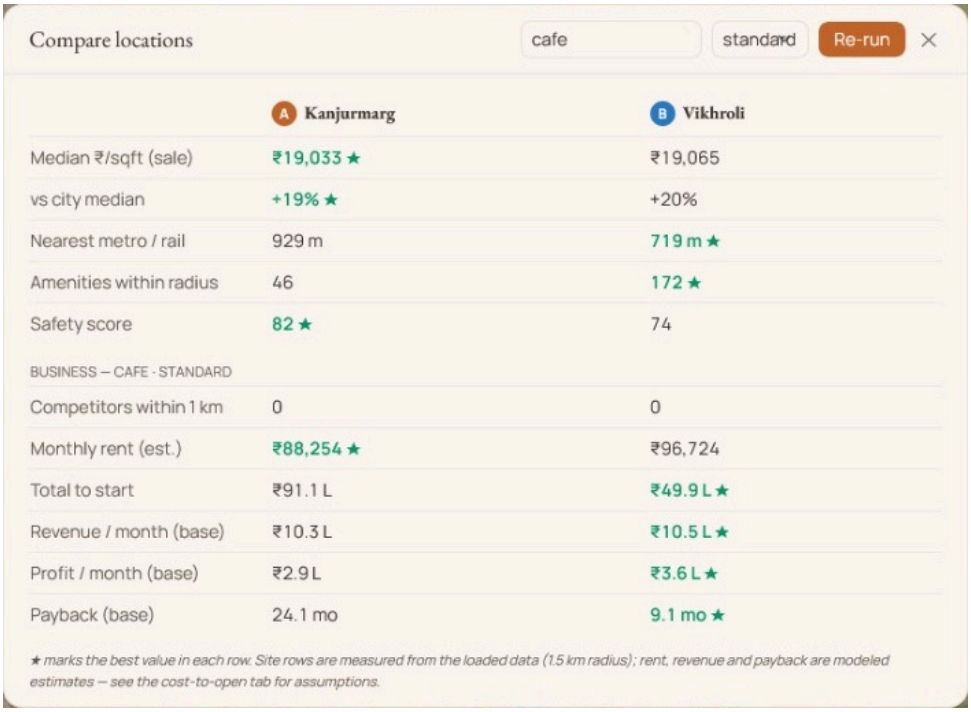
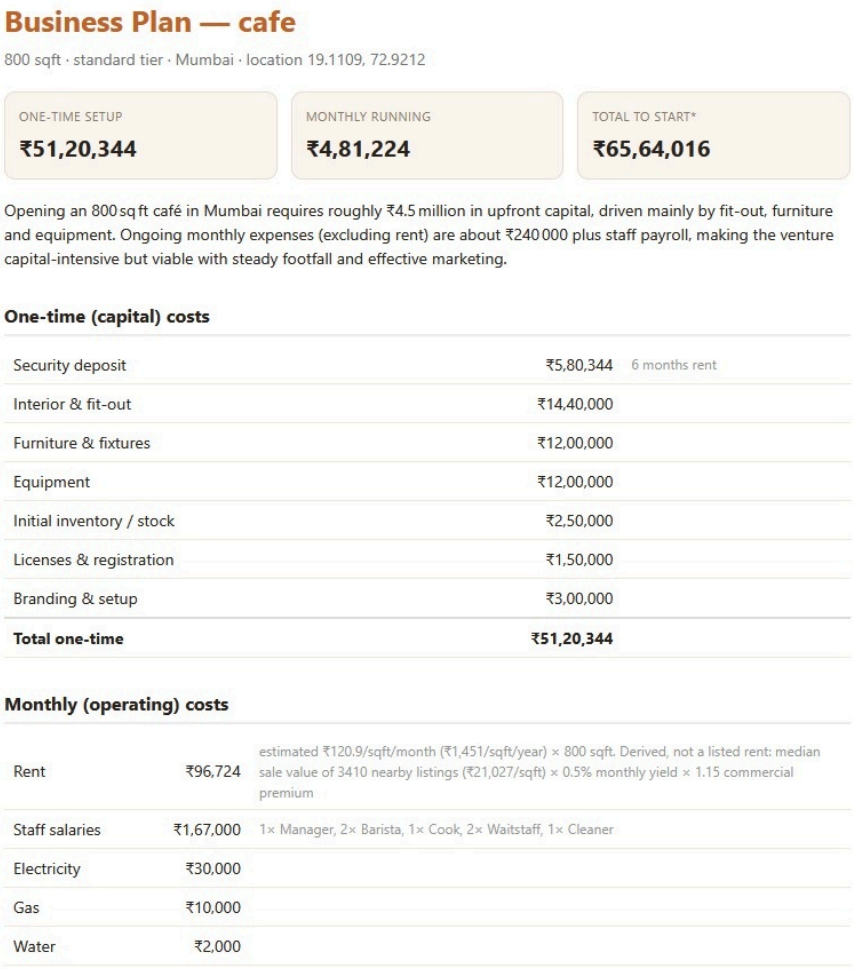


Figure 5.2: Comparison of Kanjurmarg (A) and Vikhroli (B) for a standard-tier cafe. The site rows are measured within 1.5 km. Rent, start-up cost, revenue, profit and payback are modelled estimates from a single run.

The cost-to-open tab produces a full business plan (Figure 5.3 and Figure 5.4). For an 800 square foot standard-tier cafe at Vikhroli it lists the one-time costs, the monthly costs, the capital needed to start, three revenue scenarios and the assumptions used. In this run the one-time setup comes to about 51.2 lakh rupees, the monthly running cost to about 4.8 lakh, and the base scenario gives a payback of 12.8 months. The rent line is derived from the sale prices of 3,410 nearby listings, and the report states that it is a derived value and not a listed rent.



Business Plan — cafe

800 sqft · standard tier · Mumbai · location 19.1109, 72.9212

ONE-TIME SETUP	MONTHLY RUNNING	TOTAL TO START*
₹51,20,344	**₹4,81,224**	**₹65,64,016**

Opening an 800 sq ft café in Mumbai requires roughly ₹4.5 million in upfront capital, driven mainly by fit-out, furniture and equipment. Ongoing monthly expenses (excluding rent) are about ₹240 000 plus staff payroll, making the venture capital-intensive but viable with steady footfall and effective marketing.

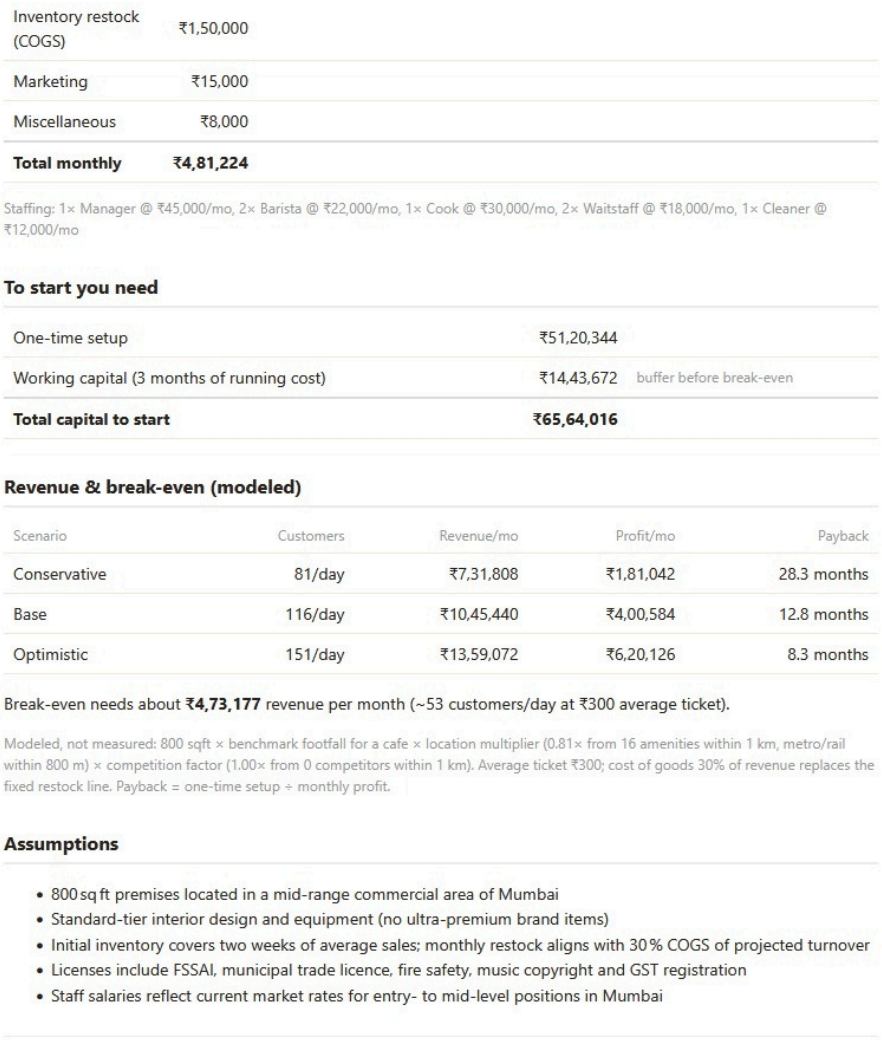
One-time (capital) costs

Security deposit	₹5,80,344	6 months rent
Interior & fit-out	₹14,40,000	
Furniture & fixtures	₹12,00,000	
Equipment	₹12,00,000	
Initial inventory / stock	₹2,50,000	
Licenses & registration	₹1,50,000	
Branding & setup	₹3,00,000	
Total one-time	**₹51,20,344**	

Monthly (operating) costs

Rent	₹96,724	estimated ₹120.9/sqft/month (₹1,451/sqft/year) × 800 sqft. Derived, not a listed rent: median sale value of 3410 nearby listings (₹21,027/sqft) × 0.5% monthly yield × 1.15 commercial premium
Staff salaries	₹1,67,000	1× Manager, 2× Barista, 1× Cook, 2× Waitstaff, 1× Cleaner
Electricity	₹30,000	
Gas	₹10,000	
Water	₹2,000	

Figure 5.3: Cost-to-open report for a cafe at Vikhroli: headline figures, one-time costs and the start of the monthly costs.



Inventory restock (COGS)	₹1,50,000
Marketing	₹15,000
Miscellaneous	₹8,000
Total monthly	**₹4,81,224**

Staffing: 1× Manager @ ₹45,000/mo, 2× Barista @ ₹22,000/mo, 1× Cook @ ₹30,000/mo, 2× Waitstaff @ ₹18,000/mo, 1× Cleaner @ ₹12,000/mo

To start you need

One-time setup	₹51,20,344	
Working capital (3 months of running cost)	₹14,43,672	buffer before break-even
Total capital to start	**₹65,64,016**	

Revenue & break-even (modeled)

Scenario	Customers	Revenue/mo	Profit/mo	Payback
Conservative	81/day	₹7,31,808	₹1,81,042	28.3 months
Base	116/day	₹10,45,440	₹4,00,584	12.8 months
Optimistic	151/day	₹13,59,072	₹6,20,126	8.3 months

Break-even needs about **₹4,73,177** revenue per month (~53 customers/day at ₹300 average ticket).

Modeled, not measured: 800 sqft × benchmark footfall for a cafe × location multiplier (0.81× from 16 amenities within 1 km, metro/rail within 800 m) × competition factor (1.00× from 0 competitors within 1 km). Average ticket ₹300; cost of goods 30% of revenue replaces the fixed restock line. Payback = one-time setup ÷ monthly profit.

Assumptions

- 800 sq ft premises located in a mid-range commercial area of Mumbai
- Standard-tier interior design and equipment (no ultra-premium brand items)
- Initial inventory covers two weeks of average sales; monthly restock aligns with 30 % COGS of projected turnover
- Licenses include FSSAI, municipal trade licence, fire safety, music copyright and GST registration
- Staff salaries reflect current market rates for entry- to mid-level positions in Mumbai

Figure 5.4: Cost-to-open report, continued: capital needed to start, modelled revenue scenarios and the assumptions used.

The plan also shows a limitation that Chapter 6 lists. The summary paragraph under the headline figures is written by the LLM, and in this run it quotes an upfront capital and a monthly expense that do not match the itemised totals below it. The itemised lines are the ones that are summed by the program; the text is not reconciled with them. The plan comes from a different run from the comparison in Figure 5.2, so its start-up cost, profit and payback differ from the Vikhroli column there. The size of that difference, with a similar rent in both runs, is consistent with the run-to-run variability of the LLM-produced line items, and it is the reason the report treats the measured site attributes as more reliable than the modelled business figures.

5.3 BENCHMARK OF THE QUERY ENGINE

5.3.1 Method

The benchmark contains 39 spatial questions in six categories: filters (10), proximity (9), named areas (7), rankings (3), listings (4) and compositional queries (6). Each question has a hand-written gold SQL query. The expected answer is the executed result of the gold query, and a predicted layer is scored by comparing geometry sets with it (exact match and F1), so column names do not matter.

Three configurations are compared. The **full engine** is the system as shipped. The **first attempt only** configuration is the same engine restricted to its first generation attempt; it is obtained by re-executing the first SQL of each full run, which makes it a paired comparison. The **direct-LLM baseline** models the proof of concept: the model names places and coordinates without a database. The baseline returns no SQL, so it is scored on grounding instead: the share of returned names that exist in the database and the share of returned points that lie near a real feature.

The results are from a single run on 7 October 2026 with `openai/gpt-oss-120b` and default sampling. The intent-check and explanation calls were skipped in the benchmark, because they do not change the SQL.

5.3.2 Accuracy of the engine

The full engine returns exactly the gold layer for 29 of the 39 questions (74.4%, geometry F1 88.4%), and every generated query executed and returned rows (Table 5.2). The repair stage was never triggered: all 39 first attempts executed and returned rows, so the first-attempt-only configuration scores identically, and this benchmark shows no benefit from repair.

Configuration	Exact match	Geometry F1	Executed
Full engine	74.4%	88.4%	100%
First attempt only (no repair)	74.4%	88.4%	100%

Table 5.2: Benchmark results on 39 questions, single run

Accuracy by category is shown in Figure 5.5. It is highest for named-area and ranking questions (both 100%) and lowest for listings (25%, one question of four). Ten questions

failed. Eight of them come from a row cap and not from a wrong query: the schema notes tell the model to use `LIMIT 500`, so questions whose gold answer is larger than 500 rows are truncated, and joined queries without `DISTINCT` let duplicate rows use up the limit. The other two are over-inclusive category choices: universities were matched together with colleges, and motorways with a pattern broader than the gold road class. We count all ten as failures, but they point to the prompt rules and to the exact-match scoring more than to a lack of spatial reasoning.

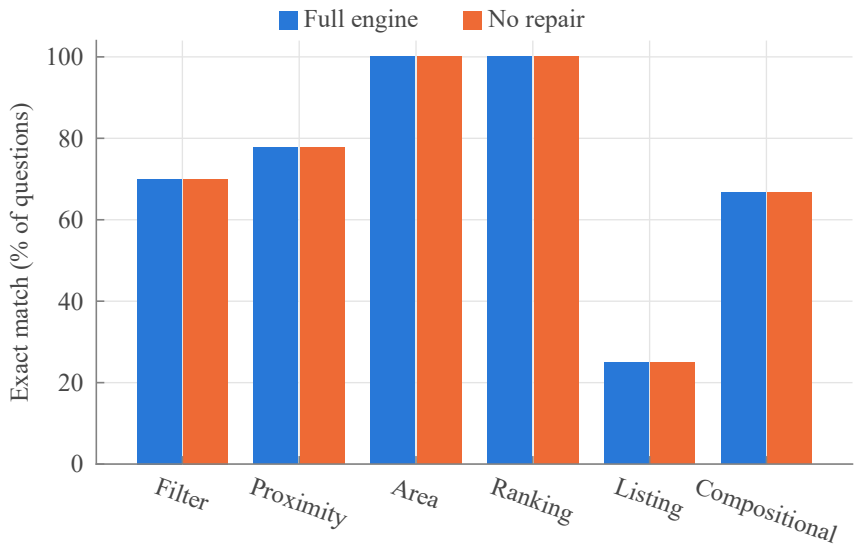


Figure 5.5: Exact-match accuracy by question category. The first-attempt-only configuration is identical to the full engine because no repair was needed.

5.3.3 The direct-LLM baseline

The direct-LLM baseline gives a very different picture (Figure 5.6). Only 19.5% of the place names it returned exist in the database, 57.0% of its coordinates lie within 150 m of some real feature, and only 10.6% lie within 200 m of a feature in the gold answer. The coordinate figure is generous, because Mumbai is dense and a random point is often near some point of interest, so the other two figures are more informative. The baseline also returns no SQL, so none of its answers can be checked.

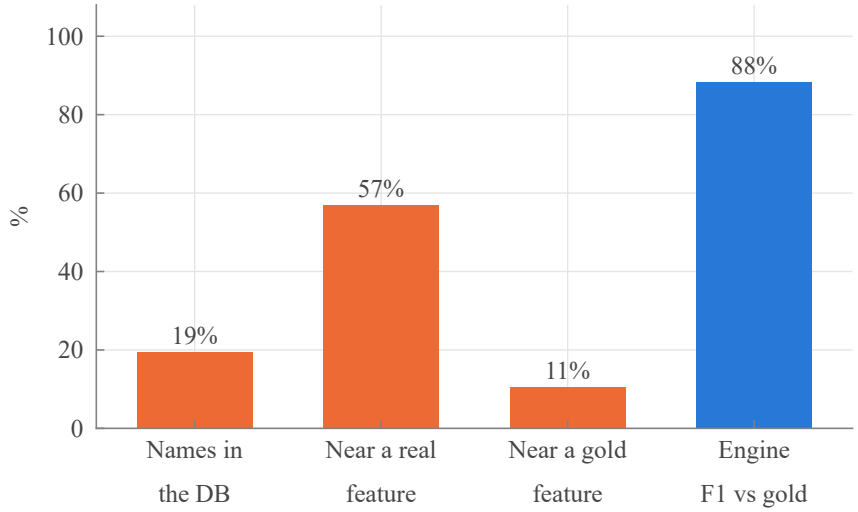


Figure 5.6: Grounding of the direct-LLM baseline (three left bars) next to the engine's geometry F1 against the gold answer (right bar). The metrics differ: the baseline bars show how much of its output is real, the engine bar its agreement with the gold layer.

5.3.4 Latency

For the full configuration in the benchmark, which includes one LLM call and the database but not the intent-check and explanation stages that the shipped engine adds, the median latency was 4.5 s and the 95th percentile 13.8 s. The direct-LLM baseline took a median of 13.2 s and a 95th percentile of 27.1 s. With 39 questions and one run, differences of a few questions should not be over-interpreted. A further ablation, in which the schema description lists only table names, is implemented in the benchmark code but was not run.

5.4 SENSITIVITY OF THE WEIGHTED RANKING

The legacy dashboard ranked areas with the weighted sum of Eqn.(3.9) and hand-picked weights. To test how much its ranking depends on those weights, we ran a sensitivity study on the 40 scored areas. We compared the hand-picked weights with equal weights, rank-order centroid (ROC) weights, AHP weights (consistency ratio 0.011), entropy weights, CRITIC weights and three scenario weight sets (budget, student and premium), and we compared the weighted sum with TOPSIS under the baseline weights. Agreement with the baseline ranking was measured by Kendall's τ and by the overlap of the top five areas. We also perturbed each weight by up to 30% one at a time, and drew 1,000 random weight vectors from a uniform Dirichlet distribution (seed 42) to see how often each area stays in the top five.

Stable under small changes to one weight. The worst-case τ for a 30% change in any single weight is 0.90 or higher, lowest for the access weight (Figure 5.8).

Not stable under a change of weighting scheme. AHP, ROC and the premium scenario keep the same top five as the baseline (τ of 0.92, 0.81 and 0.83). Equal weights, entropy weights, the student scenario, the budget scenario, TOPSIS and CRITIC share only 2, 2, 2, 1, 1 and 0 of the baseline's top five areas (τ of 0.41, 0.58, 0.87, −0.11, 0.36 and −0.01), as shown in Figure 5.7. Across random weight vectors the median τ against the baseline is 0.33 (5th to 95th percentile: −0.49 to 0.84), and no area stays in the top five in more than 45% of draws (Andheri West is highest at 44.5%).

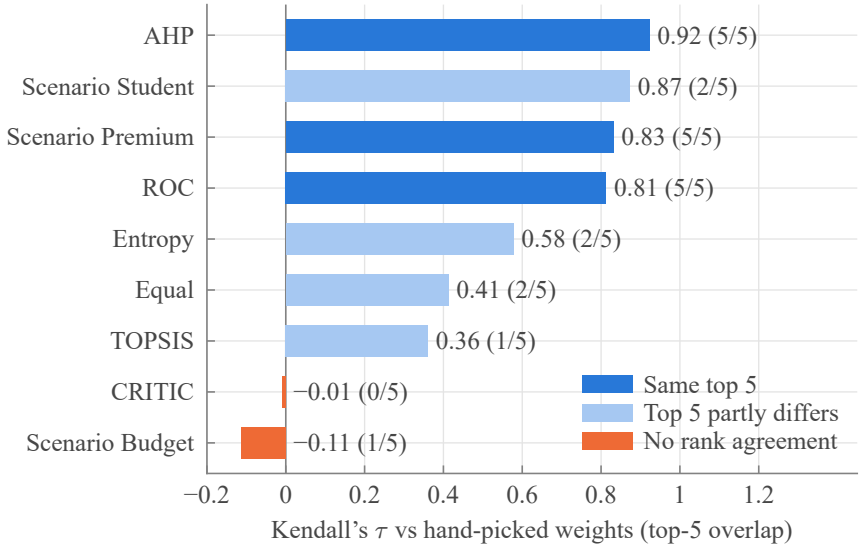


Figure 5.7: Rank agreement (Kendall's τ) of each weighting scheme with the hand-picked weights, and the overlap of the top-five areas.

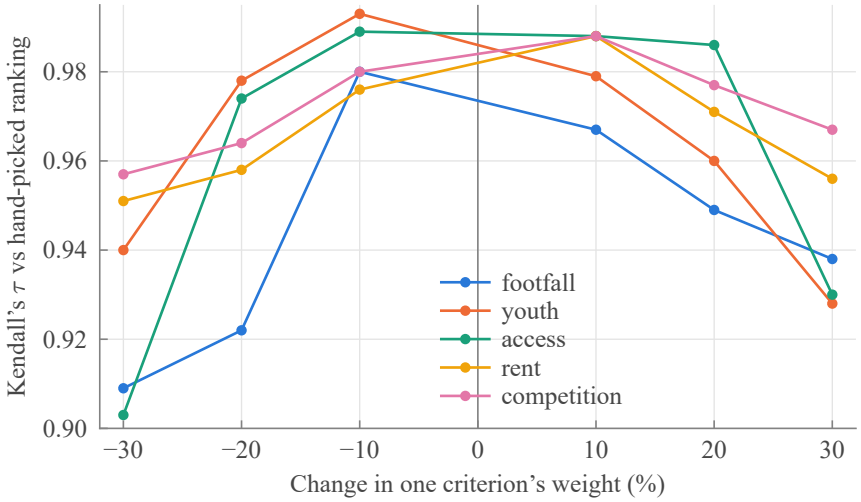


Figure 5.8: One-at-a-time sensitivity: Kendall's τ against the hand-picked ranking when one criterion's weight is changed by up to 30%, the others being renormalised.

The weights therefore encode a preference, not a neutral measurement, which is one reason the new workspace uses measured spatial queries in place of this fixed ranking. These results describe the 40-area scored dataset and should not be read as evidence about real-world site quality.

5.5 EVALUATION STATUS

Table 5.3 summarises what has and has not been measured.

Measurement	**Status**
NL-to-SQL execution accuracy on a labelled benchmark of spatial questions	39 questions, single run: 74.4% exact match, 88.4% geometry F1
Effect of the execution-based repair stage	Repair never triggered on the benchmark (0 of 39 questions), so no measurable effect
Latency of the query endpoint and the decision endpoints	Query generation step: median 4.5 s, 95th percentile 13.8 s; decision endpoints not measured
Weight sensitivity of the area ranking	Done on the 40-area dataset
Accuracy of the rent estimate against real rents	Not possible: there is no rental data

Table 5.3: What has and has not been measured

5.6 DISCUSSION

Grounding the model in a documented schema and returning the executed SQL changes the nature of the interface: answers can be checked, reproduced and corrected. The contrast with the baseline is large. When the model has to name places itself, fewer than one in five of the names it gives exist in the database, whereas the engine returns the gold layer for about three questions in four and its failures are explainable. The decision tools show that the same data supports practical questions, such as comparing three candidate shop locations on price, access, safety and competition.

The main weakness of the current work is evaluation. The benchmark is small, was run once, and did not exercise the repair stage, so we cannot say how much repair helps or

how the engine behaves on harder questions. The safeguards (a read-only role, a statement timeout, a row cap and a single-statement check) limit the damage a malformed query can do, but they have not been tested against adversarial input.

Cost and revenue estimates inherit the approximations in the rent model, the per-type benchmarks and the LLM-produced line items, and should be treated as planning aids. Because those line items are regenerated on each run, the same site can receive different start-up and payback figures. The measured site attributes (counts, distances, median prices and safety scores) are therefore more reliable than the modelled business figures, and the interface and the report both label which is which.

CHAPTER 6
CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

This project set out to build a system in which anyone can ask a spatial question in plain English and receive an answer that is computed on real data and can be checked. The system that was built does this. A question is checked for intent, translated by an LLM into a PostGIS query that is grounded in a documented schema, validated, executed under a read-only database role and, when needed, corrected. The result is shown as a layer on a vector-tile map together with a short explanation and the SQL that produced it.

On top of the query engine, four decision tools use the same database: a site report for any clicked location, a competition and catchment analysis with suggested alternative spots, a cost-to-open and break-even estimator, and a side-by-side comparison of up to three locations. They separate what is measured from what is modelled and label the second kind as an estimate. A loading pipeline driven by a city polygon fills the database from OpenStreetMap, a real-estate dataset and an area-level safety dataset, so the design is not tied to one city, although only Mumbai has been loaded.

A first benchmark of 39 spatial questions shows that the engine returns exactly the gold answer for 74.4% of them (geometry F1 88.4%), and every generated query ran. Most of its failures come from a row cap and the exact-match scoring rather than from a lack of spatial reasoning. A direct-LLM baseline that names places and coordinates without a database returned names that exist in the database only 19.5% of the time, and none of its answers could be checked. These results support the central idea of the work: that having the model write a query that is executed on real data is more reliable and more transparent than having it answer directly. A larger benchmark remains to be built.

6.2 LIMITATIONS

1. **Small benchmark.** The NL-to-SQL benchmark has 39 questions and was run once, so its accuracy figures carry substantial uncertainty, and it covers only filter, proximity, area, ranking, listing and compositional questions.
2. **Repair stage not exercised.** The engine retries after a database error or an empty result, but no query on the benchmark needed a retry, so the benefit of repair is unmeasured.
3. **Basic SQL safeguards.** Generated SQL runs under a read-only role with a statement timeout and a row cap, and is checked to be a single `SELECT`. The check is keyword-based and not a full SQL parser, and there is no table allow-list beyond the role's grants.
4. **No real rental data.** Rent is estimated from sale prices, so cost-to-open figures are approximate.
5. **Heuristic decision models.** The saturation bands, demand weights and per-type revenue benchmarks are set by judgement and have not been calibrated against real outcomes.
6. **LLM-generated cost items.** The non-rent setup and running costs, and the summary paragraph of the cost report, are written by the LLM and are not reconciled with the computed totals. Totals, profit and payback can therefore differ between runs for the same site, and the summary text can disagree with the figures beside it.
7. **Coarse listing locations.** Real-estate listings are placed near the centre of their region (228 regions) with a small deterministic offset of up to about 700 m, not at their real address.
8. **Radius-based catchments.** Catchments are circles, not travel-time isochrones.
9. **Data coverage.** Only Mumbai is loaded, and OSM completeness varies. Crime data is area-level.
10. **Model variability.** The LLM is called without a fixed temperature, so the same question can yield different SQL.
11. **No saved sites or mobile layout.** Pinned locations are not persisted, and the interface is designed for desktop.

6.3 FUTURE SCOPE

1. **Query reliability:** extend the benchmark (more questions, repeated runs, other models and harder queries that exercise the repair stage) in the style of Spider and BIRD [4], [5], [6], and remove the row-cap and over-inclusion failures found here.
2. **Safety:** replace the keyword check with parser-based validation and a table allow-list.
3. **Data:** integrate real rental data, and extend the metrics modelling with LLM and OSM features where ground truth is scarce [7].
4. **Analysis:** replace radius catchments with travel-time isochrones, and calibrate the saturation bands and revenue benchmarks against real outcomes.
5. **Cost model:** fix the LLM line items per business type and tier (or cache them per site), and generate the report summary from the computed totals, so that repeated runs agree.
6. **Usability:** saved sites and a mobile layout.
7. **Coverage:** load additional cities using the existing ETL.

BIBLIOGRAPHY

Books,

[1] J. Malczewski, *GIS and Multicriteria Decision Analysis*. New York: John Wiley & Sons, 1999.

Journal Papers,

[2] Felt, "Felt AI: AI-powered GIS and mapping platform." [Online]. Available: https://help.felt.com/felt-ai

[3] A. Mansourian and R. Oucheikh, "ChatGeoAI: Enabling Geospatial Analysis for Public through Natural Language, with Large Language Models," *ISPRS International Journal of Geo-Information*, vol. 13, no. 10, p. 348, 2024, doi: 10.3390/ijgi13100348.

[4] A. K. Kazazi *et al.*, "From Questions to Queries: An AI-powered Multi-Agent Framework for Spatial Text-to-SQL," *arXiv preprint arXiv:2510.21045*, 2025, doi: 10.48550/arXiv.2510.21045.

[5] T. Yu *et al.*, "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Brussels, Belgium: Association for Computational Linguistics, 2018, pp. 3911–3921. doi: 10.18653/v1/D18-1425.

[6] J. Li *et al.*, "Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. doi: 10.48550/arXiv.2305.03111.

[7] R. Manvi *et al.*, "GeoLLM: Extracting Geospatial Knowledge from Large Language Models," in *International Conference on Learning Representations (ICLR)*, 2024. doi: 10.48550/arXiv.2310.06213.

[8] M. Haklay and P. Weber, "OpenStreetMap: User-Generated Street Maps," *IEEE Pervasive Computing*, vol. 7, no. 4, pp. 12–18, 2008, doi: 10.1109/MPRV.2008.80.

[9] G. Boeing, “OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks,” *Computers, Environment and Urban Systems*, vol. 65, pp. 126–139, 2017, doi: 10.1016/j.compenvurbsys.2017.05.004.

APPENDIX A1
SCHEMA DESCRIPTION GIVEN TO THE SQL AGENT

This is the description of the queryable schema, with its rules, that is written into the prompt of the SQL-generating call (Chapter 3). The placeholder for the city identifier is replaced by the identifier of the city that is queried.

```
All tables are in PostGIS, geometry column is `geom`, SRID 4326. Every
feature table has
`city_id` — ALWAYS filter `city_id = :CITY_ID` on every table you read.

admin.boundary(name text, admin_level int, kind text /*city|ward*/, geom
MultiPolygon)
roads.segment(name text, highway_class text /*motorway|trunk|primary|
secondary|tertiary*/,
              maxspeed int, oneway bool, geom LineString)
transport.stop(name text, mode text /*bus|metro|rail|tram*/, geom Point)
poi.place(name text, category text, tags jsonb, geom Point)
   -- THE ONLY table of businesses/amenities. category examples:
restaurant, cafe, fast_food,
   --   bar, bank, atm, hospital, clinic, doctors, pharmacy, school,
college, university,
   --   police, fire_station, fuel, marketplace, cinema,
place_of_worship, hotel, museum, shop
realestate.listing(locality text, bhk int, type text, area_sqft float,
price float,
              price_unit text /*'Cr'=crore, 'L'=lakh*/, status text, age
text, geom Point)
demo.cell(population float, density float, age_median float, income_index
float, geom Polygon)
crime.area(name text, zone text, safety_score int /*0-100,
higher=safer*/,
              risk_level text /*low|medium|high*/, breakdown jsonb, geom
Point)
   -- ALSO a NEIGHBOURHOOD GAZETTEER: ~40 point rows named like 'Bandra
West', 'Andheri East',
   --   'Powai', 'Lower Parel'. Use it to resolve colloquial area names
to a location.
env.measurement(kind text /*aqi|flood_risk*/, value float, geom Geometry)

CRITICAL GROUNDING:
- Businesses/amenities (hospital, cafe, bank, school, pharmacy...) live
ONLY in poi.place via
  `category`. transport.stop.mode is ONLY 'bus'|'metro'|'rail'|'tram' —
NEVER an amenity.
- admin.boundary ward names are coded ('K/W Ward', 'H/E Ward') and do NOT
contain neighbourhood
  names. To find things "in <area>" (e.g. "in Bandra"), resolve the area
via crime.area:
    JOIN crime.area a ON a.name ILIKE '%Bandra%'
```

```
    AND ST_DWithin(p.geom::geography, a.geom::geography, 2000)   -- ~2 km
around the area
- "hospital" -> category IN ('hospital','clinic','doctors'); "college/
university" likewise.

RULES:
- Output ONE read-only SELECT statement. No INSERT/UPDATE/DELETE/DDL, no
semicolons chaining.
- Select a geometry column ALIASED EXACTLY `geom` (SRID 4326) plus useful
descriptive columns
  (name, category, price, safety_score, ...). The `geom` column is
required for mapping.
- Metric distance: use geography casts, e.g.
    ST_DWithin(a.geom::geography, b.geom::geography, 1000)   -- within
1000 metres
- Match names/categories with ILIKE '%...%'. When ranking, use ORDER BY +
LIMIT (<= 500).
- Never reference a table or column not listed above.
```

Listing A1.1: Schema description and rules in the SQL-generation prompt

APPENDIX A2
DATABASE DEFINITION (EXCERPT)

An excerpt of the SQL that creates the database. It shows five of the tables of Chapter 3 and the grants of the read-only role that executes generated queries.

```
CREATE TABLE core.city (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name        TEXT NOT NULL,
    country     TEXT,
    centroid    geometry(Point, 4326),
    bbox        geometry(Polygon, 4326),
    ingested_at TIMESTAMPTZ,
    status      TEXT NOT NULL DEFAULT 'pending',   -- pending | ingesting
| ready | error
    UNIQUE (name, country)
);

CREATE TABLE poi.place (
    id        BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    city_id   BIGINT NOT NULL REFERENCES core.city(id) ON DELETE CASCADE,
    name      TEXT,
    category  TEXT,                                -- restaurant | bank |
hospital | store | ...
    tags      JSONB,
    geom      geometry(Point, 4326)
);

CREATE TABLE realestate.listing (
    id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    city_id    BIGINT NOT NULL REFERENCES core.city(id) ON DELETE
CASCADE,
    locality   TEXT,
    bhk        INT,
    type       TEXT,
    area_sqft  DOUBLE PRECISION,
    price      DOUBLE PRECISION,
    price_unit TEXT,
    status     TEXT,
    age        TEXT,
    geom       geometry(Point, 4326)
);

CREATE TABLE crime.area (
    id           BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    city_id      BIGINT NOT NULL REFERENCES core.city(id) ON DELETE
CASCADE,
    name         TEXT,
    zone         TEXT,
    safety_score INT,
```

```
    risk_level   TEXT,
    breakdown    JSONB,
    geom         geometry(Point, 4326)
);

CREATE TABLE meta.schema_catalog (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    schema_name TEXT NOT NULL,
    table_name  TEXT NOT NULL,
    column_name TEXT NOT NULL,
    data_type   TEXT,
    is_geometry BOOLEAN NOT NULL DEFAULT FALSE,
    description TEXT,
    embedding   vector(768),
    UNIQUE (schema_name, table_name, column_name)
);
```

Listing A2.1: Definition of five tables

```
  GRANT CONNECT ON DATABASE geo TO geo_readonly;

  GRANT USAGE ON SCHEMA core, admin, roads, transport, poi, realestate,
  demo, crime, env, meta
      TO geo_readonly;

  GRANT SELECT ON ALL TABLES IN SCHEMA
      core, admin, roads, transport, poi, realestate, demo, crime, env,
  meta
      TO geo_readonly;
```

Listing A2.2: Grants of the read-only role used for generated queries

APPENDIX A3
BENCHMARK QUESTIONS

The 39 questions of the benchmark in Chapter 5, with the outcome of the full engine in the single run: 29 of 39 returned exactly the gold layer. Every question has a hand-written gold SQL query; the expected answer is the executed result of that query.

ID	Category	Difficulty	Question	Exact match
F01	Filter	Easy	Show all pharmacies in the city.	Yes
F02	Filter	Easy	Where are the cinemas?	Yes
F03	Filter	Easy	List all police stations.	Yes
F04	Filter	Easy	Show fuel stations.	Yes
F05	Filter	Easy	Find all universities.	No
F06	Filter	Easy	Show every hotel.	Yes
F07	Filter	Easy	Show all hospitals.	No
F08	Filter	Easy	Where are the metro stops?	Yes
F09	Filter	Easy	Show the fire stations.	Yes
F10	Filter	Easy	Show the motorways.	No
P01	Proximity	Medium	Cafes within 500 m of a metro station.	Yes
P02	Proximity	Medium	Hospitals within 1 km of a railway station.	No
P03	Proximity	Medium	Banks within 300 m of a bus stop.	No
P04	Proximity	Medium	Pharmacies within 200 m of a hospital.	Yes
P05	Proximity	Medium	Schools within 500 m of a metro station.	Yes
P06	Proximity	Medium	Metro stops within 1 km of a university.	Yes
P07	Proximity	Medium	Restaurants within 250 m of a cinema.	Yes
P08	Proximity	Medium	ATMs within 200 m of a railway station.	Yes
P09	Proximity	Medium	Bus stops within 300 m of a police station.	Yes
A01	Area	Medium	Cafes in Bandra.	Yes
A02	Area	Medium	Pharmacies in Andheri.	Yes
A03	Area	Medium	Banks in Powai.	Yes
A04	Area	Medium	Schools in Borivali.	Yes

ID	Category	Difficulty	Question	Exact match
A05	Area	Medium	Restaurants in Colaba.	Yes
A06	Area	Medium	Hospitals in Dadar.	Yes
A07	Area	Medium	Which metro stops are in Ghatkopar?	Yes
R01	Ranking	Medium	Which 5 areas are the safest?	Yes
R02	Ranking	Medium	Show the 3 least safe areas.	Yes
R03	Ranking	Medium	Top 8 areas by safety score.	Yes
L01	Listing	Medium	Listings with 3 or more bedrooms under 1 crore.	Yes
L02	Listing	Medium	Listings larger than 3000 sqft.	No
L03	Listing	Medium	Ready-to-move listings under 50 lakh.	No
L04	Listing	Medium	Listings priced above 5 crore.	No
C01	Compositional	Hard	Cafes within 500 m of a metro station that are also within 300 m of a bank.	Yes
C02	Compositional	Hard	Areas with a safety score above 70 that have a metro stop within 2 km.	Yes
C03	Compositional	Hard	Areas with no hospital within 500 m.	Yes
C04	Compositional	Hard	Schools in Bandra that are within 500 m of a bus stop.	No
C05	Compositional	Hard	Hospitals within 1 km of both a metro station and a railway station.	No
C06	Compositional	Hard	The 3 safest areas that have a metro stop within 2 km.	Yes

Table A3.1: Benchmark questions and the result of the full engine

PUBLICATIONS AND CERTIFICATES

Research paper. A paper that describes this work, “A Natural-Language Geospatial Query and Site-Evaluation System on PostGIS” (R. Chaudhary, A. Balmiki, C. Vyas and V. Wadhe), has been written from this project. **[Venue, date and status of the paper: to be filled in by the team.]**

Certificates. [Insert the scanned certificates here (paper presentation, competitions, plagiarism report and so on), one per page.]

This page is intentionally left blank

This page is intentionally left blank